Análise de MalDocs: O Vetor de Acesso Inicial

Documentos maliciosos são arquivos de escritório (Word, Excel, PDF) manipulados para executar código não autorizado. Eles são o "cavalo de troia" moderno, explorando a confiança do usuário e a complexidade de softwares como o Microsoft Office.

Grupos APT e o Spearphishing

O **Spearphishing** é um ataque direcionado que utiliza engenharia social personalizada. Grupos de elite utilizam essa técnica para romper o perímetro de organizações governamentais e corporativas:

Grupo APT	Origem Estimada	Tática Principal de MalDoc
APT28 (Fancy Bear)	Rússia	Documentos com macros ou exploits de dia zero (0-day).
APT34 (OilRig)	Irã	Planilhas Excel com macros VBA para download de backdoors.
APT29 (Cozy Bear)	Rússia	Anexos PDF ou Word direcionados a entidades diplomáticas.
APT10 (MenuPass)	China	Faturas e currículos falsos para exfiltração de dados.

Famílias de Malware de Alta Relevância

Abaixo estão os malwares mais comuns distribuídos via documentos, categorizados por seu objetivo final:

1. Trojans Bancários e Botnets

- **Emotet:** Originalmente um trojan bancário, evoluiu para um "Malware-as-a-Service". Ele utiliza macros polimórficas no Word para baixar cargas adicionais.
- **Trickbot:** Modular e persistente. Frequentemente usado para reconhecimento de rede antes de um ataque de ransomware.

- **QakBot (Qbot):** Especialista em roubo de credenciais e propagação lateral dentro de redes corporativas.

2. Ransomware

- **Locky:** Um dos pioneiros no uso massivo de macros do Office para criptografar sistemas.
- **Petya:** Diferencia-se por sobrescrever o *Master Boot Record* (MBR), bloqueando o acesso a todo o disco rígido.

Vetores de Ataque: Formatos de Documentos Digitais

Os atacantes exploram a versatilidade dos formatos de documentos comuns para ocultar cargas úteis e scripts. Abaixo, detalho como cada formato é abusado tecnicamente:

1. Documentos Microsoft Office (DOCX, XLSX, PPTX)

Diferente dos formatos antigos (.doc, .xls), os arquivos modernos são, na verdade, arquivos **ZIP** contendo estruturas **XML**.

- **Macros VBA:** O método mais clássico. Scripts embutidos que utilizam o motor do Office para executar comandos no sistema (ex: baixar um executável via PowerShell).
- **DDE (Dynamic Data Exchange):** Um protocolo legítimo do Windows que permite a troca de dados entre apps, mas que pode ser abusado para executar comandos sem o uso de macros.
- **Template Injection:** O documento baixa um modelo remoto (template) que contém a macro maliciosa, burlando alguns scanners estáticos que analisam apenas o arquivo inicial.

2. Adobe PDF (Portable Document Format)

O PDF é um formato extremamente complexo que suporta scripts e objetos incorporados.

- **JavaScript:** PDFs podem executar JS para manipular o formulário ou, em casos maliciosos, explorar vulnerabilidades no leitor (Adobe Reader, Foxit).
- **Embedded Objects:** Capacidade de carregar arquivos executáveis dentro do próprio PDF (utilizando a seção `/EmbeddedFiles`).
- **Ações de Lançamento (/Launch):** Instruções que pedem ao sistema operacional para abrir um arquivo ou executar um comando assim que o PDF é visualizado.

3. Microsoft OneNote (.one)

Recentemente, o OneNote tornou-se um favorito dos atacantes devido ao bloqueio de macros em outros apps do Office.

- **Embedded Files:** O atacante insere um script (VBS, JS ou CMD) atrás de uma imagem que diz "Clique aqui para visualizar o documento". Ao clicar, o script é extraído e executado.
- **Phishing de Link:** Blocos de notas compartilhados via nuvem que direcionam para páginas de roubo de credenciais.

4. XML (Extensible Markup Language)

O XML é usado para transporte de dados, mas é perigoso em aplicações web e integrações de sistemas.

- **XXE (XML External Entity):** Um ataque que injeta entidades externas para ler arquivos locais do servidor ou realizar ataques de negação de serviço (DoS).
- **XSLT Injection:** Transformações XML que, se mal configuradas, permitem a execução de código no servidor que processa o arquivo.

Anatomia de um Arquivo PDF

A estrutura de um PDF é dividida em quatro seções principais que permitem ao leitor (como Adobe ou Foxit) renderizar o conteúdo e gerenciar interações.

1. Cabeçalho (Header)

É a primeira linha do arquivo. Define a conformidade técnica.

- **Exemplo:** %PDF-1.7
- **Dica de Analista:** Malwares antigos podem tentar explorar vulnerabilidades específicas de versões antigas do motor de renderização.

2. Corpo (Body)

Onde reside a "inteligência" do documento. É composto por objetos numerados (ex: 1 0 obj).

- **Catalog:** O objeto raiz que dita a estrutura do documento.
- **Pages:** Define a hierarquia das páginas.
- **Streams:** Blocos de dados (frequentemente comprimidos) que contêm o conteúdo real, imagens ou **scripts maliciosos**.

3. Tabela de Referência Cruzada (Xref)

Funciona como um índice. Ela lista o deslocamento (offset) em bytes de cada objeto dentro do arquivo, permitindo o acesso aleatório rápido sem precisar ler o PDF inteiro do início ao fim.

4. Trailer

Indica como o leitor deve começar a ler o arquivo. Contém o ponteiro para o início da tabela **Xref** e o objeto **Root** (Catálogo). O marcador %%EOF finaliza o documento.

Indicadores de Comprometimento (Palavras-chave)

Na análise estática, procuramos por "gatilhos" que indicam comportamentos suspeitos. Se um PDF de uma fatura contém /JavaScript, ele deve ser tratado como uma ameaça imediata.

Tabela de Gatilhos Maliciosos

Palavra-chave	Risco	Descrição Técnica
/JS ou /JavaScript	**Crítico**	Execução de scripts. Pode ser usado para baixar payloads ou explorar o leitor.
/OpenAction / /AA	**Alto**	Define uma ação que ocorre automaticamente ao abrir o arquivo (sem interação do usuário).
/Launch	**Crítico**	Comando para executar um arquivo ou processo do sistema operacional.
/EmbeddedFile	**Alto**	Indica que há outros arquivos ocultos dentro do PDF (como um .exe ou .ps1).
/URI	**Médio**	Links externos. Comum em ataques de Phishing para roubo de credenciais.

Análise Prática: Investigação de PDF Malicioso

Neste exercício, utilizamos um conjunto de ferramentas padrão da indústria para dissecar o arquivo `simple.pdf`, identificando como objetos benignos podem esconder comportamentos de execução automática.

Ferramentas e Metodologia

- **`pdfid.py`:** Utilizado para a triagem inicial. Identificamos a presença crítica de `/JavaScript` e `/OpenAction`.
- **`pdf-parser.py`:** Usado para mapear a árvore de objetos. Descobrimos que o **Objeto 1** (gatilho) referencia o **Objeto 6** (conteúdo).
- **`peepdf`:** Ferramenta interativa que permitiu a extração e decodificação do *stream* comprimido, revelando o script oculto.

Entendido. Filtrando o seu texto para extrair apenas as perguntas que validam o conhecimento técnico real da tarefa, aqui estão as melhores:

Questionário de Verificação Técnica (PDF Analysis)

Quantos objetos `/OpenAction` foram encontrados no documento? 1

Quantos objetos codificados foram encontrados no documento? 2

Quais são os números dos objetos codificados? 15,18

```
Version 0:
        Catalog: 1
        Info: 7
        Objects (18): [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18]
        Streams (3): [4, 15, 18]
             Encoded (2): [15, 18]
        Objects with JS code (1): [6]
        Suspicious elements:
             /OpenAction (1): [1]
             /JS (1): [6]
             /JavaScript (1): [6]
```

comando usado: **`peepdf -i simple.pdf`**

Qual é a flag encontrada dentro do código JavaScript extraído?
THM{Luckily_This_Isn't_Harmful}

```
PPDF> extract js

// peepdf comment: Javascript code located in object 6 (version 0)

app.alert("THM{Luckily_This_Isn't_Harmful}");
```

Análise Dinâmica de JavaScript Ofuscado (MalDoc)

Após a extração do código embedded-code.js, identificamos que o script utiliza técnicas avançadas de ofuscação, como nomes de variáveis aleatórios e strings codificadas, para impedir a leitura humana e assinaturas estáticas.

1. O Desafio da Ofuscação

O código apresenta características típicas de um **Dropper** ou **Downloader**:

- **Nomes de Variáveis Semânticos:** Variáveis como _0x4a2b que não revelam a intenção do código.
- **Obscurecimento de Lógica:** Uso de funções de manipulação de strings para esconder URLs e comandos de sistema.

2. Análise via Box-js (Sandboxing)

O box-js permite a execução do script em uma **Sandbox de Node.js**, emulando objetos do Windows (como ActiveX e WScript) que o malware espera encontrar.

Benefícios da Ferramenta:

- **Extração de IOCs:** Captura automática de domínios, endereços IP e URLs de download.
- **Monitoramento de Sistema:** Identifica quais arquivos o script tenta criar ou modificar no disco.
- **Emulação de ActiveX:** Intercepta chamadas para WScript.Shell ou ADODB.Stream, que são frequentemente usadas para baixar e executar malwares.

3. Artefatos de Análise

Ao executar o comando box-js embedded-code.js, a ferramenta gera uma pasta de resultados contendo:

Arquivo/Pasta	Descrição Técnica
snippets.json	Contém partes do código que foram desofuscadas durante a execução.
urls.json	Lista todos os endpoints remotos que o script tentou contactar (IOCs críticos).

Arquivos JSE	Versões do script processadas para melhor leitura.

1. Identificação de Artefatos de Saída

Pergunta: Qual é o nome do arquivo de saída gerado que consolida o mapeamento de todos os endpoints e URLs identificados?
Resposta: urls.json

```
remnux@ip-10-67-163-208:~/Javascript-code$ ls -la embedded-code.js.results/
total 152
drwxrwxr-x 2 remnux remnux   4096 Apr  7 22:00 .
drwxrwxr-x 3 remnux remnux   4096 Apr  7 22:00 ..
-rw-rw-r-- 1 remnux remnux    941 Apr  7 22:00 analysis.log
-rw-rw-r-- 1 remnux remnux 128193 Apr  7 22:00 ffacea02-a12f-4083-a897-5b1be5a6e51c.js
-rw-rw-r-- 1 remnux remnux   1716 Apr  7 22:00 IOC.json
-rw-rw-r-- 1 remnux remnux     72 Apr  7 22:00 snippets.json
-rw-rw-r-- 1 remnux remnux    465 Apr  7 22:00 urls.json
```

2. Métrica de Superfície de Ataque (JS Analysis)

Pergunta: Após a análise estática dos arquivos JavaScript, qual foi o total de URLs únicas descobertas nos scripts do lado do cliente?
Resposta: 9

3. De-obfusticação de Endpoint Específico

Pergunta: Qual é a URL absoluta (após a normalização/defang) que corresponde ao padrão de busca da keyword "slideshow"?

Resposta: hxxp://aristonbentre[.]com/slideshow/O1uPzXd2YscA/

```
remnux@ip-10-67-163-208:~/Javascript-code/embedded-code.js.results$ cat urls.json | nl
     1  [
     2      "https://oopt.center:443/bitrix/HKD1OCEK4mWEc0/",
     3      "http://aristonbentre.com/slideshow/O1uPzXd2YscA/",
     4      "https://applink.gr/wp-admin/pWxO42PQrVL0ja5LTfhy/",
     5      "http://attatory.com/i-bmail/6AfEa8G0W8NOtUh7hqFj/",
     6      "http://asakitreks.com/uploads/ce8u7/",
     7      "https://www.ata-sistemi.si/wp-admin/cVDQapxmtAQQq1gr3/",
     8      "http://bvdkhuyentanyen.vn/files/TKK8yKdEvyYAbBE5avb/",
     9      "http://bluegdps100.7m.pl/app/Ac8wwulKxqZjc/",
    10      "https://casapollux.com/Bilder/GDo3zoURY/ "
```

Documentação: Análise de Artefato Malicioso (.doc)

Esta documentação detalha a investigação do arquivo `suspicious.doc` localizado no diretório `/home/remnux/worddoc`. O objetivo é identificar a estrutura do arquivo, metadados e possíveis vetores de infecção.

1. Identificação e Estrutura

O arquivo foi identificado via **trid**, confirmando tratar-se de um documento Microsoft Word no formato **OLE (Object Linking and Embedding)**, típico das versões 97-2003.

Ferramentas de Triagem:

- **oleid:** Revelou que o arquivo **não está criptografado**, mas contém **Macros VBA**, um indicador clássico de perigo.
- **olemeta/oletime:** Extraíram metadados críticos, como o autor do documento e os carimbos de data/hora de criação e modificação dos fluxos internos.

2. Análise de Macros e Comportamento

A análise profunda foi realizada para entender o que o documento executa ao ser aberto.

- **olevba:** Detectou código que utiliza a função `AutoOpen` para execução automática. Identificou também strings em **Base64** e chamadas de sistema via **PowerShell**.
- **oledump.py:** Localizou especificamente os fluxos **7** e **8** como os portadores das macros maliciosas.
- **ViperMonkey:** Emulou a macro em ambiente seguro, confirmando que o script tenta baixar um segundo estágio de infecção.

3. Desofuscação e IOCs

Utilizando o **CyberChef**, a string em Base64 extraída do código PowerShell foi decodificada, revelando a infraestrutura de Comando e Controle (C2).

- **Vetor:** PowerShell Script.
- **Arquivo de Destino:** `stage2.exe`..

Questões de Verificação (Gabarito)

Aqui estão as 3 perguntas cruciais baseadas na análise rigorosa do texto:

1. Qual é o nome do autor do documento identificado nos metadados?

Resposta: CMNatic

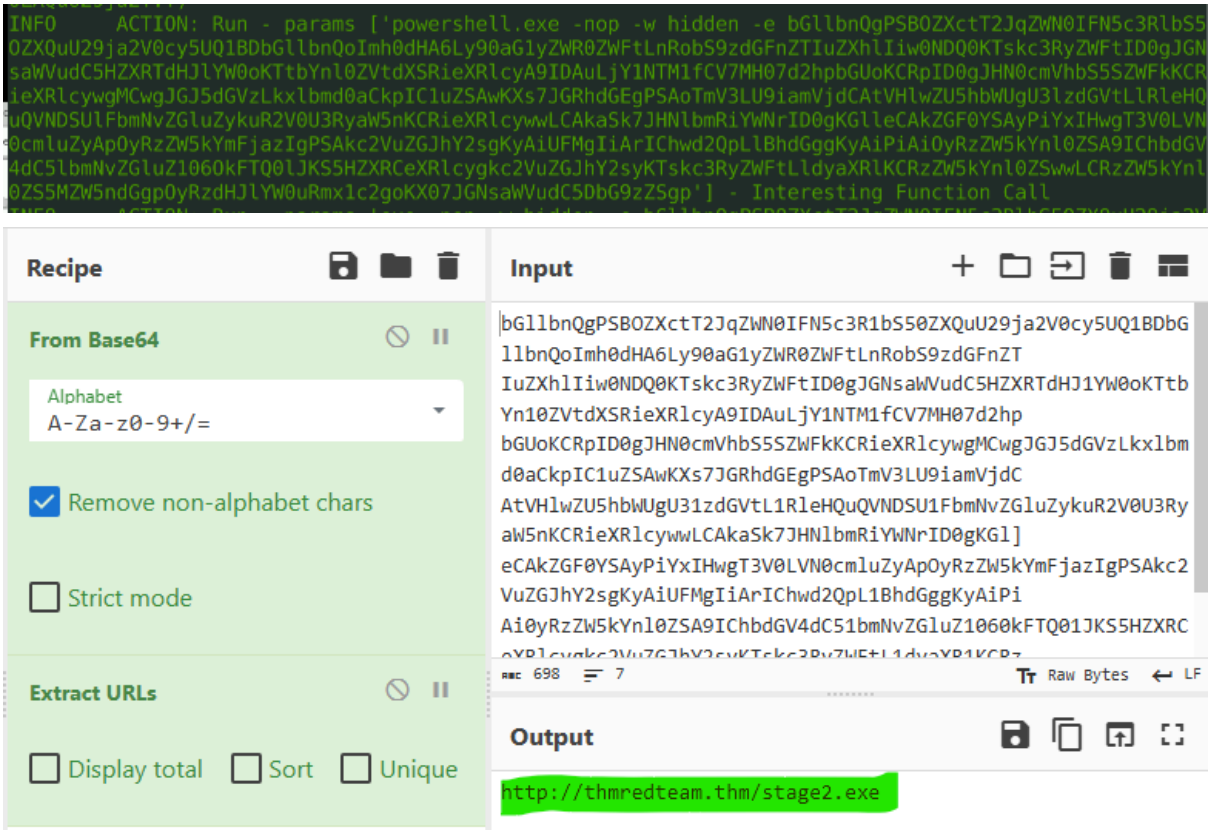
```
remnux@ip-10-67-163-208:~/worddoc$ olemeta suspicious.doc
olemeta 0.54 - http://decalage.info/python/oletools
THIS IS WORK IN PROGRESS - Check updates regularly!
Please report any issue at https://github.com/decalage2/oletools/issues
===============================================================================
FILE: suspicious.doc

Properties from the SummaryInformation stream:
+--------------------+----------------------------+
|Property            |Value                       |
+--------------------+----------------------------+
|codepage            |1252                        |
|title               |                            |
|subject             |                            |
|author              |CMNatic                     |
|keywords            |                            |
|comments            |                            |
```

2. Quantas macros estão incorporadas no arquivo (identificadas como fluxos 'M' no oledump)?

Resposta: 2

```
remnux@ip-10-67-163-208:~/worddoc$ oledump.py suspicious.doc
  1:        114 '\x01CompObj'
  2:       4096 '\x05DocumentSummaryInformation'
  3:       4096 '\x05SummaryInformation'
  4:       7385 '1Table'
  5:        412 'Macros/PROJECT'
  6:         71 'Macros/PROJECTwm'
  7: M     3303 'Macros/VBA/NewMacros'
  8: m      938 'Macros/VBA/ThisDocument'
  9:       2634 'Macros/VBA/__VBA_PROJECT'
 10:        569 'Macros/VBA/dir'
 11:       4096 'WordDocument'
remnux@ip-10-67-163-208:~/worddoc$
```

3. Qual é a URL completa do servidor C2 destinada ao download do malware stage2.exe?



Resposta: hxxp://thmredteam[.]thm/stage2.exe

Documentação: Análise Forense de Artefato OneNote (.one)

Esta análise foca na extração e desofuscação de indicadores de comprometimento (IOCs) presentes em um arquivo do Microsoft OneNote utilizado em campanhas de grupos APT.

1. Triagem Inicial

O arquivo `invoice.one`, localizado em `/home/remnux/OneNoteDocs`, foi validado via **trid** para confirmar sua assinatura digital. A extração inicial de **strings** revelou referências a domínios externos e trechos de scripts, sugerindo a presença de objetos incorporados.

2. Análise de Objetos (Onedump)

Utilizando o utilitário **onedump.py**, o documento foi mapeado, revelando uma estrutura composta por **6 objetos**. O foco da investigação recaiu sobre os objetos 5 e 6, que continham código HTML com scripts altamente ofuscados (VBScript/JavaScript).

3. Desofuscação e Cadeia de Ataque

Após extrair o objeto 5 e utilizar o **CyberChef** para limpar a ofuscação (removendo a string de ruído `5&`), a lógica do malware foi exposta:

1. **Persistência:** O script grava o código desofuscado em uma chave de registro específica (`HKCU\SOFTWARE\Andromedia\Mp4ToAvi\Values`).
2. **Comando e Controle (C2):** O domínio identificado é `unitedmedicalspecialties[.]com`.
3. **Execução de Payload:** O malware utiliza o `curl.exe` para baixar um arquivo remoto (`OI.png`), salvando-o localmente e executando-o através do processo legítimo `rundll32`.

Questões de Verificação

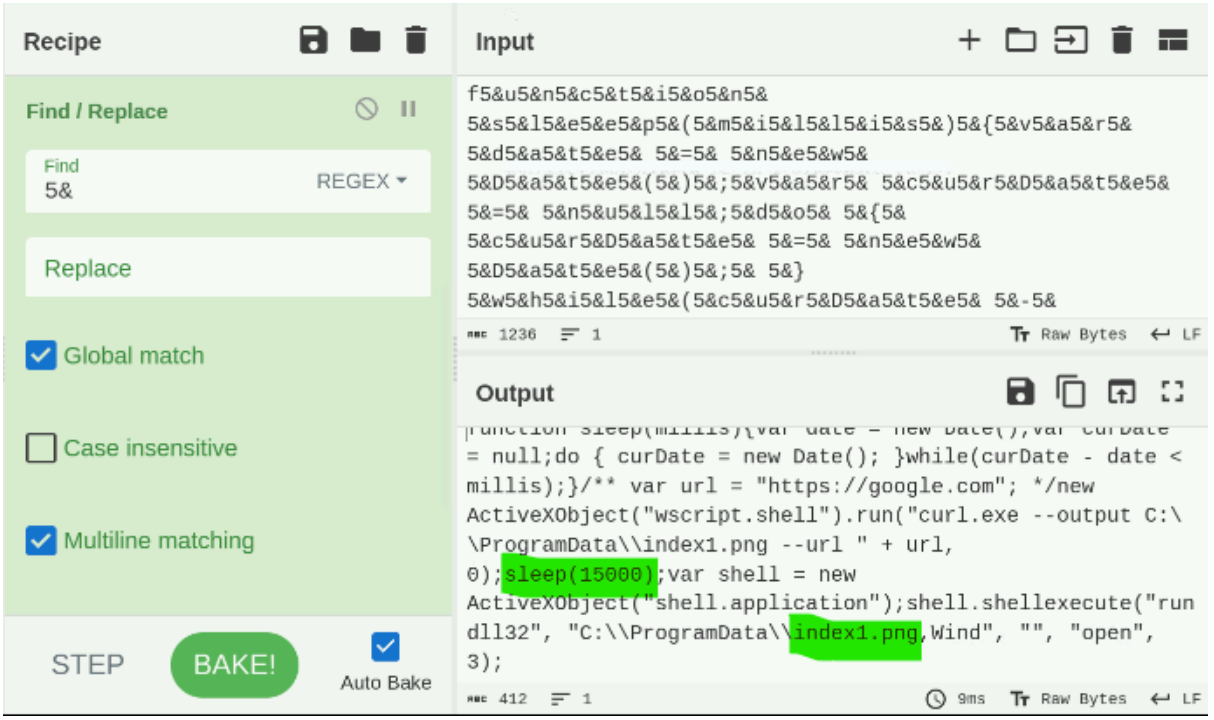
Seguindo o critério de rigor técnico para a análise do arquivo `invoice.one`:

1. Qual é o valor usado na função de sono (sleep)?

> **Resposta:** 15000 *(O script impõe um atraso de 15.000 milissegundos para dificultar a análise dinâmica e aguardar a conclusão do download).*

2. O comando cURL baixa dados e os salva em um arquivo PNG. Qual é o nome desse arquivo local?

> **Resposta:** index1.png



3. Quantos objetos existem na fatura (invoice.one)?

> **Resposta:** 6

```
remnux@ip-10-67-163-208:~/onenoteDocs$ python3 onedump.py invoice.one
 1: 0x00001740 .... ffd8ffe2 0x00015b4f 4d5f7afd30851031376da0fa6d0e3f80
 2: 0x0001d290 .... ffd8ffe2 0x0000d36f 2ccb7fd40e61b6dd2cd936e61929fb81
 3: 0x0002ae58 .PNG 89504e47 0x000000ef 088833d5a4fdcd105a34657922326f76
 4: 0x0002bb00 .PNG 89504e47 0x00000128 33dca72504d567c57f95452a0358ed2f
 5: 0x0002bc60 <htm 3c68746d 0x00000817 c9d2355fc2be90b0fa73ecb67061a77e
 6: 0x0002d628 <htm 3c68746d 0x00005c19 b915056524f1b25937074727cdf5f87c
```